okay so consider this best practices :You are a senior software architect.

You must design and refactor code using production-grade software engineering principles:

Backend architecture rules:
- Routers are HTTP-only (no business logic, no DB calls)
- Business logic must be in services/use-case layer
- All DB/API/S3 access must be in repository layer
- Domain layer must be pure logic only
- DTOs/schemas must be separated (no mixing with domain logic)
- External integrations must be isolated behind repositories/services
- Add caching layer for expensive or repeated operations with TTL + invalidation
- Ensure fallback/negative cache behavior for unavailable external services

Frontend architecture rules:
- Use feature-based architecture (NOT type-based folders)
- Pages must be thin route shells only
- Each feature owns its:
  - API layer
  - hooks
  - components
  - utils
- Use TanStack React Query for ALL server state
- Do NOT use useEffect for data fetching
- Shared components must only be truly reusable UI
- API client must be centralized and feature wrappers must be colocated in features
- Implement lazy loading at route level for performance

Data rules:

- Server state must be cached and managed via React Query

- Client state is only for UI interactions

- All backend responses must be typed and consistent with frontend expectations

Production readiness rules:

- Add loading, error, empty, and fallback states explicitly

- Make fallback data visible to users

- Avoid duplicated API logic

- Ensure role-based access control is enforced on backend

- Ensure security best practices for tokens and authentication

Code organization requirement:

- Maintain strict separation of concerns

- Maintain feature ownership boundaries

- Keep global/shared code minimal and justified

- Optimize for maintainability, testability, and scalability

okay so review all the pics in the project folfer na dthe documents as well and also refer this as well, before that for the frontend interface i want you to see the pics and make exactly like this and connect with the backend:Design and architect a full-stack, production-ready Smart Public Toilet Smoke & Hygiene Monitoring System for a college campus, based on IoT sensor integration and real-time cloud monitoring.

The system is deployed for a prototype toilet:

“IT Building Ground Floor Toilet”

This is not just a UI design task — it must describe a complete working software + IoT architecture, including backend, frontend, database, and IoT communication pipeline.

🏗️ SYSTEM ARCHITECTURE (MANDATORY)

1. IoT HARDWARE LAYER

The physical system is based on:

Raspberry Pi Pico W / Arduino-based microcontroller

MQ-2 Smoke Sensor (cigarette smoke detection)

MQ-135 Gas Sensor (air quality / hygiene monitoring)

Buzzer + LED (local alerts)

Optional OLED/LCD display

Sensors continuously monitor:

Smoke levels (cigarette detection)

Air quality / odor (hygiene condition)

IoT Firmware Logic (Embedded C / MicroPython)

Read sensor values every few seconds

Apply threshold-based detection:

Smoke detected → trigger ALERT

Poor air quality → trigger HYGIENE WARNING

Publish sensor data via MQTT to backend broker

2. MQTT COMMUNICATION LAYER (Mosquitto Broker)

Use Eclipse Mosquitto MQTT Broker as the real-time messaging backbone.

MQTT Topics:

toilet/sensors/smoke

toilet/sensors/gas

toilet/alerts

toilet/status

Flow:

IoT Device → MQTT Broker → Backend API → Frontend Dashboard

Requirements:

Use QoS level 1 for reliability

Maintain persistent connection

Support multiple sensors in future scaling

3. BACKEND SYSTEM (FASTAPI + PYTHON)

Build a scalable backend using:

FastAPI (REST + WebSocket support)

Pydantic models for validation

Async MQTT client (e.g., paho-mqtt or asyncio-mqtt)

Database: supabase

Backend Responsibilities:

A. Authentication System

JWT-based authentication

Role-based access control (RBAC)

B. MQTT Listener Service

Subscribe to sensor topics

Process incoming sensor data

Store readings in database

Trigger alerts when thresholds are exceeded

C. Real-Time API Layer

REST APIs:

/login

/register

/sensors/latest

/alerts

/maintenance/logs

/users (admin only)

WebSockets:

Real-time dashboard updates

Live sensor streaming

Instant alert notifications

D. Business Logic Engine

Detect:

Smoke event → critical alert

Gas anomaly → hygiene warning

Auto-create maintenance tickets when threshold exceeded

4. FRONTEND SYSTEM (REACT + MODERN DASHBOARD UI)

Build a modern IoT monitoring dashboard using:

React.js (or Next.js)

React Query (for API caching & syncing)

WebSockets for real-time updates

Tailwind CSS / modern UI system

UI FEATURES

🔐 Authentication Pages

Login page

Register / Create account page

Demo login buttons:

“Login as Admin”

“Login as Maintenance”

“Login as Viewer”

Clicking a demo login should instantly authenticate and redirect into the system.

ROLE-BASED DASHBOARDS

1. ADMIN DASHBOARD

Full system control panel:

Live sensor monitoring (real-time)

Analytics dashboard (charts of smoke & hygiene trends)

User management (create / delete users)

System configuration (threshold settings)

Export reports (CSV/PDF)

View all alerts & resolve them

Maintenance tracking logs

2. MAINTENANCE DASHBOARD

Operational control panel:

View active alerts in real time

Accept / resolve maintenance tickets

Update issue status

Add maintenance notes

View assigned tasks only

Cannot manage users or system settings

3. VIEWER / AUTHORITY DASHBOARD

Read-only monitoring system:

Live sensor status cards

Alert feed

Hygiene reports (weekly/monthly)

System health overview

No edit permissions

REAL-TIME DASHBOARD COMPONENTS

All roles share core components:

Live Toilet Status Card

Smoke Level Indicator

Gas / Air Quality Meter

Alert Notification Panel

Activity Timeline Log

Sensor Status Health Indicators

🛎 ALERT SYSTEM

Instant alert when:

Smoke detected

Gas threshold exceeded

Alerts should:

Appear in UI instantly (WebSocket)

Be stored in database

Be pushed to maintenance dashboard

Be logged in activity history

🧠 DATABASE DESIGN (IMPORTANT)

Use relational structure:

Tables:

users

roles

sensor_data

alerts

maintenance_logs

system_settings

🔐 AUTH FLOW

JWT authentication

Role-based route protection

Session persistence

Secure API access per role

🎨 DESIGN SYSTEM

Maintain a smart city IoT monitoring aesthetic

Style:

Minimalist dashboard UI

Clean healthcare-grade interface

Rounded cards

Soft shadows

Spacious layout

Real-time animation feel

Color Palette:

Deep teal: #0F766E

Cyan: #06B6D4

Green (safe): #22C55E

Red (alerts): #EF4444

Amber (warning): #F59E0B

White + light gray backgrounds

Typography:

Inter font family

Clean, modern spacing

🩹 DEMO MODE REQUIREMENT

System must include:

Demo Login Panel:

One-click login buttons:

Admin Demo

Maintenance Demo

Viewer Demo

No password required for demo mode.

📋 CORE SYSTEM GOAL

The system should behave like a real-time smart facility monitoring platform used in smart cities:

IoT sensors continuously stream data

Backend processes and stores events

Alerts are triggered instantly

Dashboards update in real time

Maintenance workflow is fully tracked

⚙ OPTIONAL ADVANCED FEATURES

Predictive hygiene analysis (trend-based alerts)

Sensor failure detection

Offline IoT buffering

Email/SMS alert integration

Role activity audit logs